THE OPEN SOURCE AUDIT: AN ANALYSIS OF PYTHON

A Capstone Project for the OSS NGMC Course

Prepared by**: KHUSHI GAUTAM**
Registration Number: **24BAI10126**
Course: Open Source Software (OSS)
Date: 31st March 2026
Software Audited: Python (Managed by PSF)

TABLE OF CONTENTS

- Introduction
- Part A: Origin and Philosophy
  - A1. The Problem This Software Was Created to Solve
  - A2. The PSF License vs. GPL
  - A3. The Ethics of Open Source
- Part B: Linux Footprint
- Part C: The FOSS Ecosystem
- Part D: Critical Analysis & Comparison
- Shell Script Documentation
  - Source Code & Logic
  - Execution Evidence
- Conclusion
- References

INTRODUCTION

In today's world, where cloud computing, artificial intelligence, and automation are shaping how we work and interact, Open Source Software (OSS) has grown from a small group effort into a key part of the digital economy. Among the many opensource tools, Python is a standout because of its wide use, ease of access, and adaptability across many different fields.

This report provides a detailed look at Python, a high-level, interpreted programming language that reflects the open and collaborative spirit of open-source development.
Python was chosen for this study because of its two main roles: it acts as a key language in Linux-based systems and is also a major player in data science, machine learning, and automation.

The report also examines Python's history, its licensing model, how it works with the Linux file system, and its position within the larger Free and Open Source Software (FOSS) community. By doing this, the report aims to show how Python supports modern computing and influences the global tech environment.

PART A — ORIGIN AND PHILOSOPHY

A1. The Problem This Software Was Created to Solve

To truly understand how Python developed, it's important to look at the computing world in the late 1980s. Back then, there was a big split in how people wrote software, often called the "Grand Binary." On one side were low-level languages like C, which were fast and gave a lot of control, but they were hard to use and took a lot of time to develop. On the other side were shell scripts, which were easy to learn and good for simple tasks, but they didn't have the structure needed for bigger or more complex projects, like object-oriented features or advanced data types.

Guido van Rossum, who was working at a research institute in the Netherlands called CWI, saw this gap.
He wanted to create a language that was in the middle — something that was easy to read and simple to use, but also powerful enough to handle serious programming tasks. He was inspired by a language called ABC, which was designed to be easy for beginners. But ABC had some serious drawbacks, like not being able to access files or handle errors well, which made it hard to use in real projects.

Python came about to fix these problems.
It aimed to be a middle ground, combining simplicity with power, and this set the stage for a language that would go on to play a major role in today's computing world.

The Solution:

Python was created with extensibility as a key feature from the start. The designers knew that high-level features could sometimes slow things down, so they built Python in a way that lets developers use lower-level languages like C for parts that need top performance. These parts can then be added as modules, making it easy to mix high-level code with high-performance code. This setup keeps Python simple and easy to read, but also allows it to handle performance-heavy tasks when needed.

The way Python was started shows a focus on practicality instead of being perfect in theory.
It was made to solve real problems that developers face, with a balance between being easy to use and powerful enough for complex tasks. In this way, Python isn't just a programming language—it's a carefully built tool meant to improve on what's already there, giving developers both power and flexibility.

A2. The License — The Python Software Foundation (PSF) License

Python’s license is a "Permissive" license, which stands in contrast to "Copyleft" licenses like the GPL (General Public License).

The Four Freedoms Analysis:

1. Freedom 0 (Run): Python can be run for any purpose, from academic research to high-frequency trading on Wall Street.
2. Freedom 1 (Study): The CPython source code is hosted on GitHub, allowing anyone to study the "Global Interpreter Lock" (GIL) or memory management systems.
3. Freedom 2 (Redistribute): Users can bundle Python with their own software and sell it.
4. Freedom 3 (Improve): Anyone can submit a "PEP" (Python Enhancement Proposal) to suggest changes to the language.

GPL vs. PSF License: The Critical Difference

The GNU General Public License (GPL) is sometimes called "viral" or "copyleft," which means that if you use code licensed under GPL, you have to share your software under the same open-source terms. This keeps derived works open, but it can also make it harder for proprietary systems to use the software.

On the other hand, the Python Software Foundation (PSF) License is more flexible.

Companies like Apple or Google can use Python inside their systems, change it as needed, and aren't required to share their changes. This openness makes Python more attractive to businesses, helping it get used more widely in different industries.

Because of this, Python has received strong support from companies and is used in both open-source and proprietary setups.

This has led to ongoing development, more resources, and financial backing for the Python Software Foundation, which helps keep Python strong in the global tech world.

A3. The Ethics of Open Source

The ethical basis of open-source software is often explained using the "Cathedral vs. the Bazaar" model. In this analogy, proprietary software is like a Cathedral—built carefully by a small group of experts in a controlled and private setting. On the other hand, open-source development is similar to a Bazaar—a lively, decentralized, and collaborative environment where many people contribute, try new ideas, and create things together. While the Bazaar may seem less organized, it encourages quick innovation, openness, and teamwork in solving problems.

The Debate Around Open Knowledge:

A major ethical issue in open-source communities is whether it's fair for companies to make money from software created by a worldwide group of volunteers.

In the Python world, this is handled through an unspoken "social contract." Even though companies use open-source tools for profit, they often give back to the community by supporting development, funding projects, and hiring key contributors to work on Python full-time. This creates a mutual benefit where businesses and the open-source community both gain, helping to keep things running smoothly and improving over time.

Learning from Past Achievements:

A core ethical value in the Python community is the focus on using existing work and working together to move forward.

Developers usually don’t start from zero; instead, they use well-established tools and frameworks like Django for web apps and TensorFlow for AI. This reflects a belief in not wasting time on repeated efforts and in helping everyone progress together. By building on what others have done, the open-source community speeds up innovation, allowing software development to move much faster than many traditional methods.

PART B — LINUX FOOTPRINT

The Resident Interpreter

Python isn't just an external program in Linux systems; it's actually a key part of the operating system itself.

In Linux versions like Ubuntu and Fedora, Python is used in many important system tasks.

For example, during system maintenance, tools like the apt package manager use Python scripts to handle software installation, updates, and managing dependencies.

Also, in automation tasks like Linux startup (like Cloud-Init) and installation tools (like Anaconda), Python is used to set up and manage system configurations and deployments. This shows how important Python is for system setup and running.

Key Directories:

- Binary: /usr/bin/python3 — This is the main file that runs Python scripts on the system.
- Standard Library: /usr/lib/python3.10/ — This folder holds Python's built-in modules, often called the "batteries included" approach. It includes important libraries like os, sys, and math.
- Third-Party Packages: /usr/local/lib/python3.x/dist-packages/ — This is where externally installed libraries are kept, usually added through tools like pip. These libraries add extra features beyond what's in the standard library.

Update Model:

Python uses a "rolling stability" release model.

New minor versions (like from Python 3.10 to 3.11) are released every year, offering new features and performance improvements. Security updates and fixes are also provided as needed to fix any problems.

In Linux, the system maintainers are responsible for handling these updates.

They make sure that Python upgrades are well tested and integrated safely, so that system stability isn't affected and important processes and apps keep working properly.

PART C — THE FOSS ECOSYSTEM (Units 3 & 4)

Python and the LAMP Stack

Traditionally, the "P" in the LAMP stack (Linux, Apache, MySQL, PHP/Perl) referred to scripting languages such as Perl or PHP.
However, in modern computing environments, Python has become a popular alternative because of its flexibility, easy-to-read syntax, and a wide range of tools and libraries available.

Python works well with the LAMP stack.
It runs on Linux, which provides a stable and scalable foundation. For web services, Python connects smoothly with Apache and Nginx using standardized protocols like WSGI (Web Server Gateway Interface) and ASGI (Asynchronous Server Gateway Interface). These protocols allow both traditional and asynchronous web applications to be deployed effectively. Additionally, Python interacts with databases such as MySQL and PostgreSQL using established tools like psycopg2, which makes it easier to handle data and develop server-side functionality.

Community and Forks

Python’s open-source nature has led to the creation of different versions of the language suited for various purposes.
Some notable examples are:

- MicroPython: A compact version of Python made for small devices and environments with limited resources, especially in IoT (Internet of Things) projects.

- PyPy: Another version of Python that focuses on running faster by using Just-In-Time (JIT) compilation, making it ideal for demanding computing tasks.
- CircuitPython: A version of MicroPython designed for education and simple hardware projects, often used in robotics and beginner-level electronics.

These variations highlight Python's versatility, showing how it can be adapted for use in everything from small embedded systems to big data and complex applications.

The Python language is managed by a Steering Council, which helps make decisions in a balanced and fair way.
This system prevents any one company from taking control of Python, keeping it open, collaborative, and driven by the community. Because of this, Python keeps growing and changing in ways that match both new technology and the needs of its large global group of developers.

PART D — OPEN SOURCE VS PROPRIETARY (Critical Analysis)

Dimension	Python (Open Source)	MATLAB (Proprietary)
Security	Highly transparent; code is audited by thousands of eyes.	Security relies on the vendor's internal audits (Security by Obscurity).
Cost	Free for everyone.	Licenses cost thousands of dollars per user.
Support and Reliability	Community forums, StackOverflow, and GitHub.	Dedicated corporate support and documentation.
Freedom to modify	You own your code and the tool used to run it.	You are "renting" the right to use the software.
Verdict	Python wins on accessibility and innovation.	MATLAB wins on specialized, "out-of-the-box" engineering tools.
Community vs Corporate Control	Governed by the non-profit PSF and community.	Entirely controlled by MathWorks.

SHELL SCRIPT DOCUMENTATION

Script 1: System Identity Report

Concepts Used: Command substitution, Variables, `/etc/os-release` parsing. Logic: This script uses the `uname` and `uptime` commands to provide a snapshot of the environment where Python is running.

CODE:

File Edit Selection View Go Run Terminal Help Search

$ script1.sh M ×

C: > Users > Dell > Desktop > new pro > OSSproject > $ script1.sh

```bash
#!/bin/bash
# Script 1: System Identity Report
# Author: Khushi | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Khushi"
SOFTWARE_CHOICE="Python"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
USER_HOME=$HOME
UPTIME=$(uptime -p)
DISTRO=$(grep "PRETTY_NAME" /etc/os-release | cut -d'=' -f2 | tr -d '"')
DATE_TIME=$(date)

# --- Display ---
echo "========================================"
echo "      System Identity Report            "
echo "========================================"
echo " Open Source Audit - $STUDENT_NAME"
echo "----------------------------------------"
echo "Distro Name   : $DISTRO"
echo "Kernel Version: $KERNEL"
echo "Current User  : $USER_NAME"
echo "Home Directory: $USER_HOME"
echo "System Uptime : $UPTIME"
echo "Current Date  : $DATE_TIME"
echo "----------------------------------------"
echo "My Chosen Software: $SOFTWARE_CHOICE"
echo "Note: This Operating System is covered by an Open Source License (likely GPL)."
echo "========================================"
```

OUTPUT:

```
$ bash script1.sh
script1.sh: line 13: uptime: command not found
grep: /etc/os-release: No such file or directory
==========================================
      System Identity Report
==========================================
 Open Source Audit - Khushi
------------------------------------------
Distro Name   :
Kernel Version: 3.6.6-1cdd4371.x86_64
Current User  : Dell
Home Directory: /c/Users/Dell
System Uptime :
Current Date  : Tue Mar 31 00:09:41 IST 2026
------------------------------------------
My Chosen Software: Python
Note: This Operating System is covered by an Open Source License (likely GPL).
==========================================
```

Script 2: FOSS Package Inspector

Concepts Used: `if-then-else`, `case` statements, `dpkg` integration. Logic: This script demonstrates "conditional logic" by checking the system's package database to see if Python is installed.

CODE:

script2.sh M

C: > Users > Dell > Desktop > new pro > OSSproject > $ script2.sh

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Khushi | Course: Open Source Software

PACKAGE="python3"

# Check if package is installed
# Works for both Debian/Ubuntu (dpkg) and RHEL/Fedora (rpm)
if command -v $PACKAGE >/dev/null 2>&1; then
    echo "FOSS Package: $PACKAGE is installed."

    # Try to get version info
    if command -v dpkg >/dev/null 2>&1; then
        # Fixed awk command: one line to avoid string termination errors
        dpkg -l | grep -E "^ii  $PACKAGE " | awk '{print "Version: " $3 " | Summary: " $4 " " $5}'
    elif command -v rpm >/dev/null 2>&1; then
        rpm -qi $PACKAGE | grep -E "Version|License|Summary"
    fi
else
    echo "$PACKAGE is NOT installed."
fi

# Logic for software description
case $PACKAGE in
    python3)
        echo "----------------------------------------"
        echo "Software: Python 3"
        echo "Purpose : A versatile high-level programming language."
        echo "Mission : Designed for readability and ease of use."
        echo "Open Source: Managed by the Python Software Foundation (PSF)."
        ;;
    *)
        echo "No information found for $PACKAGE."
        ;;
esac
echo "----------------------------------------"
```

OUTPUT:

```
$ bash script2.sh
FOSS Package: python3 is installed.
-----------------------------------------
Software: Python 3
Purpose : A versatile high-level programming language.
Mission : Designed for readability and ease of use.
Open Source: Managed by the Python Software Foundation (PSF).
-----------------------------------------
```

Script 3: Disk and Permission Auditor

Concepts Used: `for` loops, Array iteration, `awk` for string parsing.
Logic: This script iterates through an array of critical system paths to audit security permissions and disk usage.

CODE:

script2.sh M $ script3.sh M ×

: > Users > Dell > Desktop > new pro > OSSproject > $ script3.sh

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Khushi | Course: Open Source Software

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================="
echo "      Directory Audit Report             "
echo "========================================="

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Getting the directory's permissions and size
        # ls -ld: displays info for the directory itself
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Owner: $OWNER | Size: $SIZE"
    else
        echo "Directory $DIR does not exist on this system."
    fi
done

echo "-----------------------------------------"
echo "Searching for the Python software location..."
# which -a displays all paths for the command
PYTHON_LOC=$(which python3)
if [ -n "$PYTHON_LOC" ]; then
    echo "Python is located at: $PYTHON_LOC"
    ls -l "$PYTHON_LOC"
else
    echo "Python binary not found."
fi
echo "=========================================="

```

OUTPUT:

```
$ bash script3.sh | fold -w 50
==========================================
      Directory Audit Report
==========================================
/etc => Permissions: drwxr-xr-x | Owner: Dell | Si
ze: 1.6M
Directory /var/log does not exist on this system.
Directory /home does not exist on this system.
/usr/bin => Permissions: drwxr-xr-x | Owner: Dell
| Size: 91M
/tmp => Permissions: drwxr-xr-x | Owner: Dell | Si
ze: 1.1G
------------------------------------------
Searching for the Python software location...
Python is located at: /c/Users/Dell/AppData/Local/
Microsoft/WindowsApps/python3
lrwxrwxrwx 1 Dell 197121 121 Mar 11 17:44 /c/Users
/Dell/AppData/Local/Microsoft/WindowsApps/python3
-> /c/Program Files/WindowsApps/Microsoft.DesktopA
ppInstaller_1.28.220.0_x64__8wekyb3d8bbwe/AppInsta
llerPythonRedirector.exe
==========================================
```

Script 4: Log File Analyzer

Concepts Used: `while read` loops, Positional parameters (`$1`, `$2`), `grep`.
Logic: This script processes large text files line-by-line, demonstrating the efficiency of shell scripting for diagnostic tasks.

CODE:

script3.sh M script4.sh M ×

C: > Users > Dell > Desktop > new pro > OSSproject > $ script4.sh

```bash
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Khushi | Course: Open Source Software
# Usage: ./script4.sh /var/log/syslog "error"

# --- Input Handling ---
# Use the first argument as the log file, or default to syslog
LOGFILE=${1:-"/var/log/syslog"}
# Use the second argument as the keyword, or default to "error"
KEYWORD=${2:-"error"}
COUNT=0

# Check if the log file exists and is readable
if [ ! -r "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found or not readable."
    echo "Usage: ./script4.sh [LOGFILE] [KEYWORD]"
    exit 1
fi

echo "========================================"
echo " Analyzing Log File: $LOGFILE"
echo " Searching for Keyword: $KEYWORD"
echo "========================================"

# Reading the file line-by-line
while IFS= read -r LINE; do
    # -i: case insensitive, -q: quiet (only returns exit code)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "----------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE."
echo "Last 5 matches found:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
echo "========================================"
```

OUTPUT:

```
scripts$ ./script4.sh
==================================
 Log Analyzer Report
==================================
Keyword 'install' found 1894 times in /var/log/dpkg.log
```

Script 5: Manifesto Generator

Concepts Used: User input (`read`), String concatenation, File redirection (`>>`)

Logic: An interactive script that creates a persistent record of the user's philosophy, demonstrating how automation can interact with human users.

CODE:

```
$ script3.sh M    $ script4.sh M    $ script5.sh M ×
C: > Users > Dell > Desktop > new pro > OSSproject > $ script5.sh
#!/bin/bash
# Script 5: The Open Source Manifesto Generator
# Author: Khushi | Course: Open Source Software

echo "=========================================="
echo "      Welcome to the Manifesto Generator   "
echo "=========================================="
echo "Answer three questions to create your open source philosophy."
echo ""

# read -p displays a prompt to the user
read -p "1. Which open source tool is most important to you? (e.g., Python, Linux, VSCode): " TOOL
read -p "2. What is the most important freedom in open source? (e.g., Modify, Share, Study): " FREEDOM
read -p "3. Why do we need open source software in the future? (one sentence): " REASON

# date formatting: %d %B %Y = Day Full-Month Year
DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)
OUTPUT="manifesto_${USER_NAME}.txt"

# Creating the manifesto content
# > overwrites the file, >> appends to it
echo "------------------------------------------" > $OUTPUT
echo "      Open Source Manifesto - $DATE       " >> $OUTPUT
echo "------------------------------------------" >> $OUTPUT
echo "I, $USER_NAME, believe that $TOOL is essential for progress." >> $OUTPUT
echo "The freedom to $FREEDOM allows software to evolve beyond corporate limits." >> $OUTPUT
echo "In the future, open source matters because $REASON." >> $OUTPUT
echo "------------------------------------------" >> $OUTPUT
echo "Generated by Khushi's OSS Script." >> $OUTPUT

echo ""
echo "Manifesto saved to: $OUTPUT"
echo "Content of your manifesto:"
cat $OUTPUT
echo "==========================================="
```

OUTPUT:

```
$ bash script5.sh
==========================================
     Welcome to the Manifesto Generator
==========================================
Answer three questions to create your open source philosophy.

1. Which open source tool is most important to you? (e.g., Python, Linux, VSCode): VSCode
2. What is the most important freedom in open source? (e.g., Modify, Share, Study): Modify
3. Why do we need open source software in the future? (one sentence): it promotes innovation, security , affordability and global collboration by allowing anyone to use , improve and share technology freely

Manifesto saved to: manifesto_vaish.txt
Content of your manifesto:
------------------------------------------
     Open Source Manifesto - 30 March 2026
------------------------------------------
I, vaish, believe that VSCode is essential for progress.
The freedom to Modify allows software to evolve beyond corporate limits.
In the future, open source matters because it promotes innovation, security , affordibility and global collboration  by allowing anyone to use , improve and share technology freely.
------------------------------------------
Generated by Shabi's OSS Script.
==========================================
```

CONCLUSION

This audit shows that Python is much more than just a programming language; it is a strong example of open-source values in action. Starting from its creation to solve real-world problems for developers, Python has grown into a widely used standard around the world. It shows how teamwork and innovation can lead to big changes.

Even though Python is used a lot in businesses and commercial projects, it has always stayed true to its core beliefs like openness, simplicity, and community involvement.

Its popularity proves that ethical practices in open-source can go hand in hand with technology progress and economic success. In the end, Python reminds us of the importance of honesty, sharing knowledge, and working together to shape the future of computing.

REFERENCES

Python Official Documentation
https://docs.python.org/3/
Python Software Foundation
https://www.python.org/psf/ The
Cathedral and the Bazaar
(for open-source ethics concept) GNU
General Public License
https://www.gnu.org/licenses/gpl-3.0.en.html
Linux Documentation
https://www.kernel.org/doc/ Apache HTTP
Server
https://httpd.apache.org/
MySQL
https://dev.mysql.com/doc/ Django
https://www.djangoproject.com/